

AIPL に express mode と where を入れるとどうなるか

— ABCL/1 へ近づけることの利点と代償 —

ABCL^c+ / AIOS 処理系 (yaskodama/abclcp)

2026 年 07 月 30 日 09:39 JST 版

概要

先の比較レポートで、AIPL は ABCL/1 から三種のメッセージ送信を継承しつつ express mode、返信先の一級性、where 制約つきパターン受信を落としていることを整理した。本稿はそのうち **express mode** と **where** の二つを AIPL に入れた場合に何が起きるかを、構文・実装・型システム・形式的保証の四層で検討する。

結論を三点にまとめる。第一に、**両者は型システムをほとんど壊さない**。どちらも「いつメッセージを受理するか」の話であり、「どんな型を持つか」の話ではないためである。第二に、**where は進行定理に第四の枝「ガード待ち」を追加する**。これは失うものが明確で、しかも **select** と名前 dispatch の並立という既存の不整合を解消できるので、費用対効果が高い。第三に、express mode は意外にも**型健全性の証明を壊さない**。形式モデル AIPL⁻ はアクターの逐次性を最初から落としており、同一アクターへの複数タスクの並行実行を既に許しているからである。壊れるのは**プログラム側の不変条件**であり、だからこそ ABCL/1 は atomicity 宣言を用意していた。

一方で express mode には実装上の前提がある。現在のメソッドのローカル変数はアクターの環境に直接置かれ、呼び出し時に退避・復元されていない。これは今は観測できない潜在的な問題だが、入れ子実行を導入した途端に表に出る。本稿はこれを実測で確認した。

目次

1	はじめに	1
1.1	問い	1
1.2	なぜこの二つなのか	2
2	提案する形	2
2.1	where — 状態依存の受理条件	2
2.2	express mode — 割り込み	3
3	実装への影響	3
3.1	where の実装	4
3.2	express mode の実装	4
3.3	前提となる修正 — ローカル変数のスコープ	4
4	型システムへの影響 — ほぼ無傷	5
4.1	どちらも「型」の話ではない	5
4.2	where の型付け	5

4.3	express mode における reply の帰属	6
4.4	ガードの純粋性という新しい要求	6
5	形式的保証への影響	6
5.1	保存定理 — 影響なし	6
5.2	進行定理 — where は枝を一本増やす	6
5.3	逐次性 — express mode は証明を壊さない	7
5.4	危ういのはプログラム側の不変条件	7
6	メリット	7
6.1	where の利点	7
6.2	express mode の利点	8
7	デメリットと危険	8
7.1	where の代償	8
7.2	express mode の代償	9
7.3	二機構の相互作用	9
8	段階的な導入案	9
9	結論	10
9.1	どちらを先に入れるべきか	10
9.2	全体としてどう評価するか	10

1 はじめに

1.1 問い

AIPL (ABCL^c+) は ABCL/1 の子孫を自認しているが、ABCL/1 の中核機構のうち次の三つを持っていない。

- **express mode** — 処理中のメッセージを中断して割り込む機構。
- **返信先の一級性** — 返信先を値として他者へ渡し、委譲する機構。
- **where** — オブジェクトの状態に依存した受理条件。

本稿は前者と後者、すなわち**受理と割り込みに関する二つ**を導入した場合の帰結を検討する。返信先の一級化は線形型を要する別問題なので扱わない。

1.2 なぜこの二つなのか

現在の AIPL には次の具体的な穴がある。

- **now** で返信を待っているアクターは、その間**他のメッセージを一切処理できない**。1 アクター 1 スレッドで、`await_future` が条件変数でブロックするためである。状態を問い合わせることも、止めることもできない。

- 状態に依存した待ち合わせが書けない。「バッファが空でないときだけ `get` を受理する」を表現するには、受理してから条件を見て、駄目なら自分に送り直すしかない。受信規律の外へ出てしまう。
- 受信経路が二本ある。名前による `dispatch` と `select` が並立し、同名のメッセージが通常の `dispatch` に先取りされると `select` には残らない。

`express mode` は一つ目、`where` は二つ目と三つ目に直接効く。

2 提案する形

2.1 `where` — 状態依存の受理条件

メソッド宣言と `select` の `case` に、真偽値のガードを付けられるようにする。

```
class Buffer {
  var n = 0;
  var cap = 3;

  // n < cap のときだけ put を受理する
  method put(x) : unit where n < cap {
    n = n + 1;
    print("put, n=" ++ n);
  }

  // n > 0 のときだけ get を受理する
  method get() : int where n > 0 {
    n = n - 1;
    reply(n);
  }
}
```

`select` 側も同じ形にする。

```
select {
  case get() where n > 0 -> { reply(n); }
  timeout 500 -> { print("empty"); }
}
```

意味は ABCL/1 と同じである。ガードが偽のメッセージは受理されず、メールボックスに残る。状態が変わって真になれば受理される。

2.2 `express mode` — 割り込み

送信側は三種それぞれに `express` 版を用意する。既存の `send!` (`UnsafeSend`) と衝突しないよう、修飾子として `express` を置く。

```
express send a.stop();           // 割り込んで実行、待たない
var s = express now a.status();  // 割り込んで実行、返信を待つ
var f = express future a.status();
```

受信側は、割り込みで実行されてよいメソッドを明示する。どのメソッドが他のメソッドの実行を中断しうるかは、読み手にとって最も重要な情報だからである。

```
class Worker {
  var progress = 0;
  var stopped = 0;

  // 通常のメソッド。長く走る
  method run(n) : unit {
    var i = 0;
    while i < n do {
      if (stopped >= 1) { i = n; }
      progress = progress + 1;
      i = i + 1;
    }
  }

  // 割り込みで呼ばれてよい。run の実行中でも応答する
  express method status() : int {
    reply(progress);
  }

  express method stop() : unit {
    stopped = 1;
  }
}
```

割り込まれたくない区間は `atomic` で保護する。

```
method transfer(k) : unit {
  atomic {
    from = from - k;    // この2行の間に割り込まれると
    to   = to + k;      // 合計が壊れる
  }
}
```

3 実装への影響

AIPL の実装を読んだうえで、変更箇所を具体的に見積もる。

3.1 where の実装

ファイル	変更
lexer.mll	<code>where</code> をキーワードに追加
parser.mly	<code>method_decl</code> に省略可能な <code>where expr</code> 、 <code>select_case</code> にも同じものを追加
ast.ml	<code>method_decl</code> に <code>guard : expr option</code> 、 <code>select_case</code> にも同様
infer.ml	ガードをメソッドの環境（フィールド＋仮引数）で <code>bool</code> として型検査するだけ
eval_thread.ml	ここが本体 。actor_loop が <code>Queue.pop</code> で先頭を取るのをやめ、メールボックスを走査して「名前が一致し、かつガードが真」の最初のメッセージを選ぶ

走査の機構はすでにある。`select` の実装 `pop_matching_message` はメールボックスを一旦すべて取り出し、パターンに合うものを探し、残りを戻している。ガード評価を足すだけで再利用できる。

注意 1. これは `select` と通常 `dispatch` を統一する好機でもある。通常 `dispatch` がガード付きの走査になるなら、`select` は「この名前の集合のみを対象に、タイムアウト付きで走査する」という特別な場合にすぎない。受信経路が一本になり、先取り競合という現在の不整合が消える。

3.2 express mode の実装

ファイル	変更
lexer.mll	<code>express</code> 、 <code>atomic</code> をキーワードに追加
parser.mly	送信の <code>express</code> 版 3 種、 <code>express method</code> 、 <code>atomic { ... }</code> 文
ast.ml	送信に <code>mode</code> を持たせる、 <code>method_decl</code> に <code>express : bool</code> 、 <code>Atomic of stmt</code>
types.ml / infer.ml	<code>express</code> メソッドの <code>reply</code> を、選択されたメッセージの ρ に束縛する（ <code>select</code> の <code>case</code> で既に行っている処理と同型）
eval_thread.ml	アクターに第二のキューを足す。 <code>eval_stmt</code> の文境界で <code>express</code> キューを覗き、あれば入れ子で評価する。 <code>atomic</code> 区間ではこの覗き込みを止める

割り込みは協調的なポーリングで実現する。OCaml のシステムスレッドを外から割り込むのは現実的でないので、`eval_stmt` が文の境界で `express` キューを確認し、あればその場で `express` メソッドの本体を入れ子に評価して戻る。粒度は文単位になる。

3.3 前提となる修正 — ローカル変数のスコープ

入れ子実行を入れる前に直さなければならない箇所がある。

測定 1. 現在、メソッドの仮引数は呼び出しの前後で退避・復元されている。

```
let saved = List.map (fun p -> (p, Hashtbl.find_opt actor.env p)) params in
  List.iter2 (fun p v -> Hashtbl.replace actor.env p v) params arg_vals;
  ...
  eval_stmt actor mdecl.body;
```

```
List.iter (fun (p, ov) -> ...) saved      (* 復元 *)
```

しかし本体の `var` で宣言されたローカル変数は `eval_stmt` が `actor.env` へ直接書き込むだけで、退避も復元もされない。

これは今は観測できない。型検査器が各メソッドを独立した環境 (`clone env`) で検査するため、他のメソッドのローカルを参照するプログラムは型エラーになる。実測すると次のとおりである。

```
class C {  
  method a() : unit { var tmp = 777; print("a: tmp=" ++ tmp); }  
  method b() : unit { print("b: tmp=" ++ tmp); } // 宣言していない  
}
```

```
TYPE_ERROR: line 8, col 24: unbound variable: tmp
```

型が守っているなので実害が出ていない。しかし `express mode` を入れると同一アクター上で入れ子実行が起きる。外側のメソッドが `var tmp` を持ち、割り込んだ `express` メソッドも `var tmp` を持てば、両者は型検査上まったく別の変数なのに実行時には同じ `actor.env` のエントリを共有し、上書きされる。

したがって `express mode` の前提として、メソッド呼び出しごとのフレームを導入する（またはローカル変数も仮引数と同様に退避・復元する）修正が必要である。これは `express mode` とは独立に価値のある修正でもある。

4 型システムへの影響 — ほぼ無傷

ここが本稿の中心的な観察である。

4.1 どちらも「型」の話ではない

`where` も `express mode` も、メッセージをいつ受理するかを変える機構であって、メッセージがどんな型を持つかを変えない。AIPL の型システムが述べているのは後者だけである。

現在の検査	二機構の影響
メソッドの存在	影響なし
引数の個数と型	影響なし
返信の型	影響なし
返信の回数（高々一度）	<code>express</code> は文脈の分離が必要。 <code>select</code> の <code>case</code> で既に実装済みの処理と同型
<code>select</code> の照合	<code>where</code> でガードの <code>bool</code> 検査が増えるだけ
公開時の注釈	影響なし

4.2 `where` の型付け

ガードはメソッドの環境（フィールド＋仮引数）で `bool` として型検査するだけである。追加規則は次の一行で足りる。

$$\frac{\Gamma, \bar{x}:\bar{\tau} \vdash g : \text{bool} \quad \Gamma, \bar{x}:\bar{\tau} \vdash_{C.m} \text{body}}{\Gamma \vdash \text{method } m(\bar{x}) \text{ where } g \{ \text{body} \}} \text{ METHOD-GUARD}$$

依存型や篩型は**不要**である。それらが必要になるのは「ガードが真なら本体は安全」を**証明**したい場合であって、型安全性のためではない。

4.3 express mode における reply の帰属

express メソッドの本体にある `reply` は、割り込んだ express メッセージへの返信である。囲んでいた通常メソッドの返信ではない。

これは既に解決済みの問題と同型である。`select` の case 本体の `reply` も、囲むメソッドではなく選択されたメッセージへの返信であり、型検査器は case 本体で `reply` をそのメソッドの ρ に束縛し直している。express メソッドも同じ扱いでよい。実行時に `ctx_msg_id` を退避・復元する処理も、`select` の実装がそのまま雛形になる。

4.4 ガードの純粋性という新しい要求

一点だけ、型システムに新しい要求が生じる。**ガードは副作用を持ってはならない**。受理の可否を試すためにガードを評価するので、そこで状態が変わると「試したら受理条件が変わった」ことになる。

現在の OCaml 実装には効果システムが無い。最小限の対処は構文的な制限である。

- ガードで参照できるのはフィールドと仮引数のみ。
- 呼び出せるのは純粋なプリミティブのみ（比較、算術など）。
- `now / future / send / reply / new / become` は書けない。

これは既存の構文検査 (`expr_has_reply` など) と同じ形の走査で実装できる。

5 形式的保証への影響

5.1 保存定理 — 影響なし

保存 (preservation) は「well-typed な構成が一步進んでも well-typed」である。状態の型不変条件は「状態が常に宣言された型を持つ」ことしか言わない。`where` は遷移が起こる条件を狭めるだけ、express mode は遷移の順序を変えるだけなので、どちらも保存を壊さない。

5.2 進行定理 — where は枝を一本増やす

現在の進行定理は三択として述べられている — 値・簡約・`await` 待ち。デッドロックは型では排除できないので、その境界を明示したものである。

`where` を入れると、**第四の枝が必要になる**。メールボックスにメッセージがあるのに、どのガードも真でないため配送できない構成が存在するからである。

現在	<code>where</code> 導入後
値である	値である
簡約できる	簡約できる
<code>await</code> 待ちである	<code>await</code> 待ちである
	ガード待ちである

これは失うものが明確である。定理の主張が一つ弱くなる代わりに、「なぜ止まっているか」の分類が一つ増える。言い換えれば、**where** は**デッドロックの形を一つ増やす**。バッファが空のまま `get` だけが溜まれば、それは正当な待ちであるが、プログラムの誤りでそうなった場合と静的に区別できない。

5.3 逐次性 — `express mode` は証明を壊さない

これは直観に反するが、確認できた事実である。形式モデル AIPL⁺ は「保証しないこと」の一覧で、アクターの逐次性を明示的に**落として**いる。

actor の逐次性 — 落とした。同じ actor あての二通が同時にタスクになりうる。

配送規則は無条件に新しいタスクを起こすので、同一アクターに対して複数のタスクが並行に走りうる。そして同レポートは、これが**型安全性には影響しない**と明言している。状態の型不変条件は状態が常に正しい型であることしか言わず、どの順で書き換えられるかは問わないからである。

したがって `express mode` の割り込みは、**形式モデルが既に許している実行の一部**にすぎない。逆に言えば、現在の**実装**が形式モデルより厳しく（1 アクター 1 スレッドの逐次実行）作られているだけである。`express mode` はその差を埋める方向の変更であり、型健全性の証明をやり直す必要はない。

5.4 危ういのはプログラム側の不変条件

壊れるのは型ではなく、プログラムが持っている**意味的な不変条件**である。

リポジトリの健全性レポートは哲学者の食事問題について資源デッドロック自由を証明しているが、その証明はフォーク割り当てプロトコルの抽象状態機械 $s = (\text{phase}, \text{own})$ についてのものであり、状態の健全性 $\text{wf}(s)$ は「Hold なら `lo` のみを持つ」のように **phase と own が整合していること**を要求する。

`express mode` で「`phase` を書き換えたが `own` はまだ」という瞬間に割り込まれると、その瞬間の状態は $\text{wf}(s)$ を満たさない。型は壊れないが、抽象状態機械との対応が壊れる。同レポートが「実装との対応は言えない」と述べている箇所が、まさにここで効いてくる。

これが **ABCL/1 に atomicity 宣言があった理由**である。割り込みを許すなら、割り込まれてはいけない区間を宣言できなければならない。`express mode` と `atomic` は分離できない一対の機構である。

6 メリット

6.1 `where` の利点

1. **同期が宣言的になる**。有界バッファ、資源プール、段階制御（初期化が済むまで受け付けない）などが、受理条件としてそのまま書ける。フラグ変数も待ちループも要らない。
2. **「受理してから条件を見て送り直す」という悪い定石が消える**。現在この形を書くと、条件が偽の間メールボックスを自分で埋め続けることになる。受信規律の中で待てるようになる。
3. **受信経路が一本化できる**。通常 `dispatch` がガード付き走査になれば、`select` はその特別な場合になる。現在の「`select` が通常 `dispatch` に先取りされる」という観測可能な不整合が消える。これは表現力を増やす話ではなく、後付けで生じた矛盾の解消である。
4. **既存の機構を再利用できる**。メールボックスの走査は `select` の実装にすでにある。ガード評価を足すだけである。
5. **型システムをほとんど触らない**。ガードの `bool` 検査が増えるだけで、依存型は要らない。

6.2 express mode の利点

1. 塞がったアクターに手が届く。現在 `now` で待っているアクターは石になる。express mode があれば、待っている最中でも状態問い合わせ・中止指示・監視に応答できる。これは現在の AIPL の最大のモデル的な穴に直接効く。
2. デッドロックからの脱出手段になる。相互 `now` で固まった二者に、外から `express send a.abort()` を送れる。型では排除できないデッドロックに対する運用上の答えになる。
3. 監視・内省が現実的になる。`aio_*` の内省プリミティブは、対象アクターが動いていると意味のある値を返しにくい。express mode があれば「いま何をしているか」を当人に聞ける。
4. タイムアウトの段階的な扱いが書ける。`select` の `timeout` は自分が待つ側の機構だが、express mode は相手を叩く側の機構である。両方あると「一定時間返らなければ問い合わせ、それでも駄目なら中止」が書ける。
5. 型健全性の証明を壊さない (§5.3)。

7 デメリットと危険

7.1 where の代償

項目	内容
進行定理が弱まる	「ガード待ち」という第四の枝が増える (§5.2)
FIFO でなくなる	メッセージの処理順が到着順と一致しなくなる。意味論として明記が必要
飢餓	あるメッセージのガードが永遠に真にならない場合、そのメッセージは永久に残る。公平性は保証できない
静的に検出できない誤り	恒偽のガードを持つメソッドは「決して受理されない」が、型検査は通る。 <code>select</code> の <code>arity</code> 不一致と同じ種類の静かに壊れる誤りが一つ増える
コスト	配送ごとにメールボックスを走査し、各メッセージについてガードを評価する。 $O(1)$ から $O(n)$ になる
ガードの純粋性	効果システムが無いので構文的に制限する必要がある (§4.4)

飢餓と恒偽ガードについては、緩和策がある。明らかに恒偽なガード (`false` や定数比較) は警告できる。また、あるメッセージが一定回数の走査で受理されなかったことを実行時に記録すれば、診断の手がかりになる。

7.2 express mode の代償

項目	内容
意味的不変条件が壊れる	最大の問題。 <code>atomic</code> と対で導入しなければ使い物にならない (§5.4)
前提となる修正	メソッド呼び出しごとのフレームが必要。現在ローカル変数は <code>actor.env</code> に直接置かれ、退避・復元されていない (§3.3、実測済み)
割り込み粒度	協調的ポーリングなので文境界でしか割り込めない。単一の長い式や組込み関数の内部では割り込めない
再入	<code>express</code> メソッドが通常メソッドを呼ぶ、 <code>express</code> の中で <code>express</code> が来る、といった入れ子の深さ制限が要る
デバッグ可能性	実行の非決定性が増える。「どこで割り込まれたか」を再現するのが難しい
既存プログラムの前提	「メソッドは中断されない」を暗黙に仮定している既存コードは、 <code>express</code> メソッドを一つ足した時点でその仮定を失う。既定は非割り込みにすべき
証明との対応	型健全性は無傷だが、抽象状態機械との <code>refinement</code> を主張していた箇所は <code>atomic</code> の裏付けが必要になる

7.3 二機構の相互作用

- **express メッセージにガードを許すか。**許すと「割り込みたいのに受理されない」という状況が生じ、`express` の目的（緊急の応答）に反する。**express メソッドにはガードを付けられない**とするのが妥当である。
- **atomic 区間の中で now を書けるか。**書けると、`atomic` のまま返信を待つことになり、その間割り込めない。`express mode` の目的を損なう。**atomic 区間では now と await を禁止**するのが安全である。これは構文的に検査できる。
- **ガード評価中に割り込むか。**ガードは純粹で短いので、割り込まないのが単純である。

8 段階的な導入案

いずれも独立に価値があり、この順序なら各段階で退行を測れる。

段階	内容	備考
0	メソッド呼び出しフレームの導入	ローカル変数を <code>actor.env</code> から出す。express mode の前提だが単独でも意味がある潜在バグの修正
1	<code>select</code> の case に <code>where</code>	最小。通常 dispatch の FIFO を変えないので影響が閉じている。ガードの <code>bool</code> 検査と純粋性検査をここで作る
2	メソッドに <code>where</code> 、通常 dispatch を走査化	進行定理に枝が増える。 <code>select</code> を統一する好機
3	<code>atomic</code> 区間	express より先に入れる。中で <code>now / await</code> を禁止する検査も同時に
4	express mode	段階 0 と 3 が前提。既定は非割り込み、 <code>express method</code> と明示したものだけ割り込める

段階 1 と 2 の間、段階 3 と 4 の間で、既存コーパスの型検査判定と差分テストハーネスを回して測るべきである。これまでの改修で、見積りが実測と食い違うことが繰り返しあった。

9 結論

9.1 どちらを先に入れるべきか

`where` を先に入れるべきである。理由は三つある。

1. 型システムへの影響が `bool` 検査の追加だけで済む。
2. 失うものが「進行定理に枝が一本増える」と明確に述べられる。
3. `select` と名前 dispatch の並立という**既存**の不整合を解消できる。表現力の追加ではなく、負債の返済としての価値がある。

express mode は、前提となる修正（呼び出しフレーム）と対で必要な機構（`atomic`）を抱えており、しかも既存コードの暗黙の仮定を壊す。価値は高いが、`where` のあとに置くのが順序として自然である。

9.2 全体としてどう評価するか

この二つを入れると、AIPL は ABCL/1 に確かに近づく。そして**その代償は型システムではなく、推論の難しさとして現れる。**

型健全性は無傷である。保存は影響を受けず、逐次性は形式モデルが最初から落としているので express mode も証明を壊さない。進行定理だけが `where` によって枝を一本増やす。

しかし、プログラムについて**個別に証明していた性質**—— 哲学者の食事問題の資源デッドロック自由のような —— は、割り込みが入ると抽象状態機械との対応を `atomic` で裏づけ直す必要がある。

先の比較レポートで「失った表現力が型付けを容易にしている」と書いた。本稿はその裏返しである。**表現力を戻すと、型は保てるが、個別の証明の手間が増える。**どちらを重く見るかが判断の分かれ目であり、研究目的が「実装された言語の健全性を機械検証する」ことにあるなら、`where` までは進み、express mode は `atomic` の設計が固まってから、というのが妥当な線である。

参考文献

- [1] Akinori Yonezawa, Jean-Pierre Briot, Etsuya Shibayama. Object-Oriented Concurrent Programming in ABCL/1. *OOPSLA 1986*, pp. 258–268.
- [2] Akinori Yonezawa (ed.). *ABCL: An Object-Oriented Concurrent System*. MIT Press, 1990.
- [3] 本リポジトリ docs/aipl_vs_abcl1_ja.pdf — AIPL と ABCL/1 の機能・モデル比較。
- [4] 本リポジトリ docs/aipl_soundness_ja.pdf — AIPL⁺ の型健全性・型安全性の Coq 検証、哲学者の食事問題、および「保証しないこと」の一覧。
- [5] 本リポジトリ docs/aipl_reply_inference_ja.pdf — reply からの戻り値型推論と select における返信の帰属の扱い。
- [6] 本リポジトリ docs/aipl_guide_ja.pdf — 現状の言語ガイド。